

Assignment 2: Building a Visual Artificial Neural Network

Ulric Aird

Department of Computer and Technology, St. George's University

Introduction to Artificial Intelligence (COMP 300)

Professor Senthilkumar Somasundaram

October 16th, 2026

Assignment 2: Building a Visual Artificial Neural Network

With the task of creating a Visual Artificial Neural Network, I've decided to build one using Python. I chose a dataset of students' grades from a program I built as a side project. The data set includes at least 69 students, with at least each student having grades for each subject for each term. Using this data, I created a prediction model to analyze and predict the possibility of a student being promoted or retained.

In any common model, there are 3 layers: Input Layers, Hidden Layers, and Output Layers. In the input layer, the number of neurons used, depends on the number of input data that is allowed, or if no limit is set, it will use all data as input data, which in this case is 69 students. The hidden layers are a different case, where instead of having a set number of fixed hidden layers, I've decided to make the program choose its own number of hidden layers depending on the amount of input data. In the program, the rule states that if the input data amount is less than 20, it chooses 1 hidden layer; if it's less than 60 input data, it chooses 2 hidden layers; if it's less than 150 input data, it chooses 3 hidden layers; and anything more than 150, it chooses 4 hidden layers. Each time a hidden layer is created, it is always followed by an activation function called "ReLU" which means "Rectified Linear Unit," which is a widely used activation function. The output layer of the model will always be one. The image below shows a diagram of the neural network and an activation graph of the hidden layer.

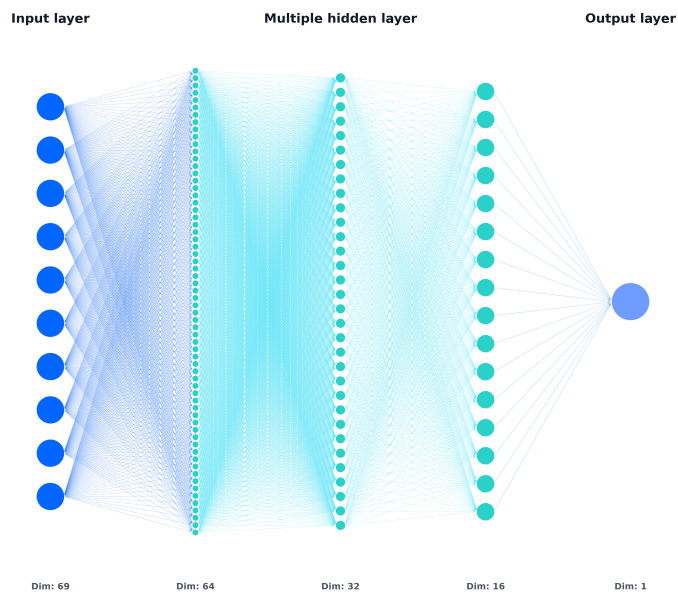


Image 1: This is the data set of 69 students as mentioned.

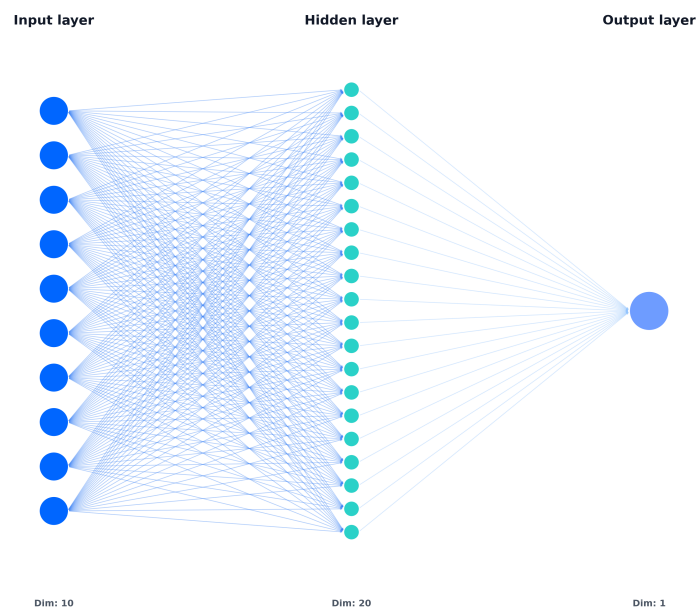


Image 2: This shows the amount of input data we wanted selected (10).

The word “Dim” in the images is short for dimensions, which also means the number of neurons in each layer based on the set of defined instructions.

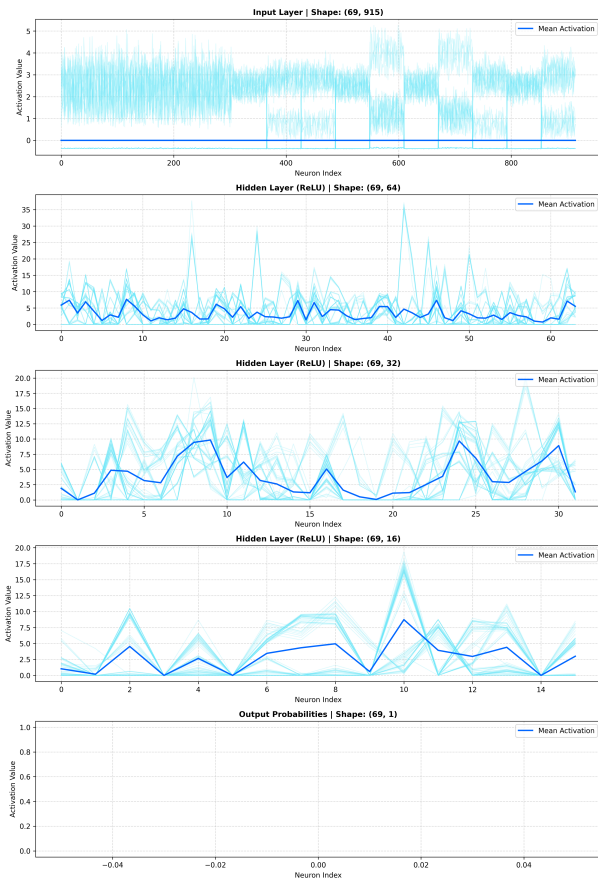


Image 1: Shows activation function graph starting with input data amount being 69 students

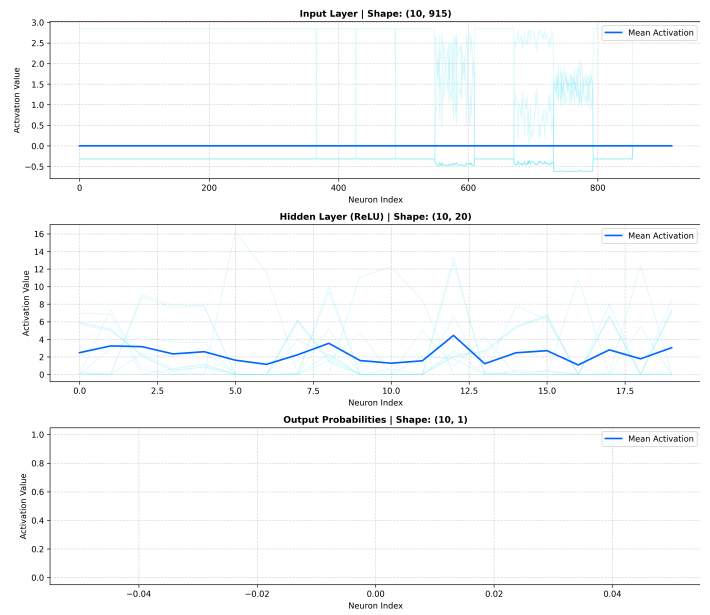


Image 2: Shows the activation function graph starting with input data amount being 10 students when selected

In the model, the epoch is set to 100, which means that it's a training loop that passes over the data, the set number of times in which it was defined, in a sequence. While training the model, we performed data splitting where we set the test size to 20%, which means that the model never sees this percentage of data during training until all 100 epochs / training loops are finished. This is done to prevent biased behavior and performance when predicting the outcome. I also implemented train losses, which will be shown in a diagram, where it will tell you how wrong the model is in predicting the outcome of the students. If the loss value is high, then the prediction outcome will be worse, and if the loss value is low, then the prediction outcome is closer to being accurate.

I also added optimizer which acts "guide" during the model training. When the 1st training loop starts, everything is random because it doesn't have any initial training, but as the training continues, it adjusts the weights of the model to produce the correct prediction.

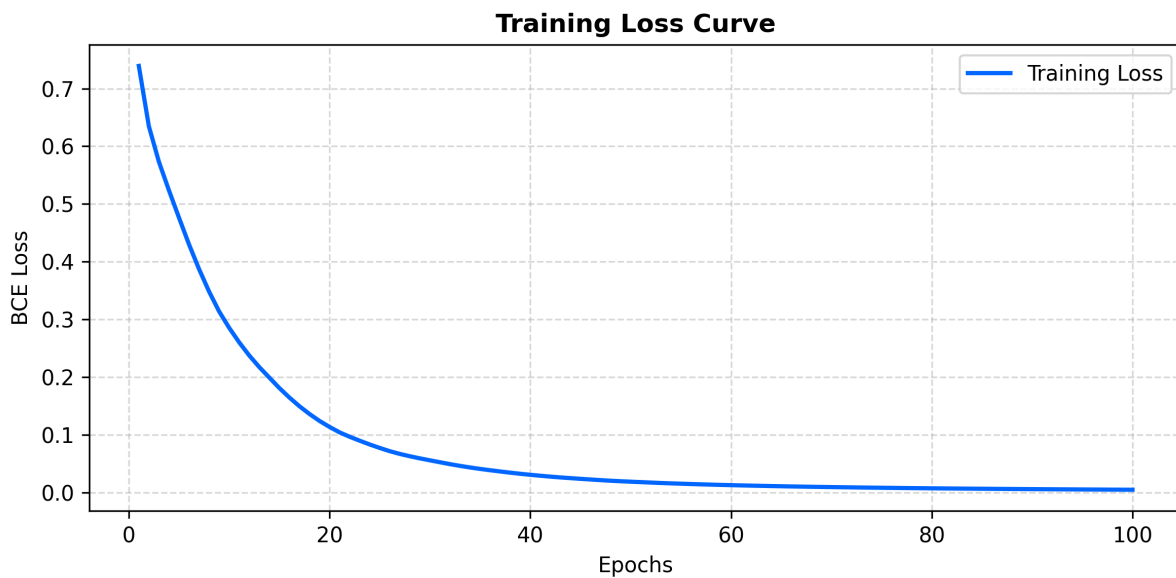


Image 1: Training Loss from the first training loop to the final training loop where it's using all the input data totaling to 69 students

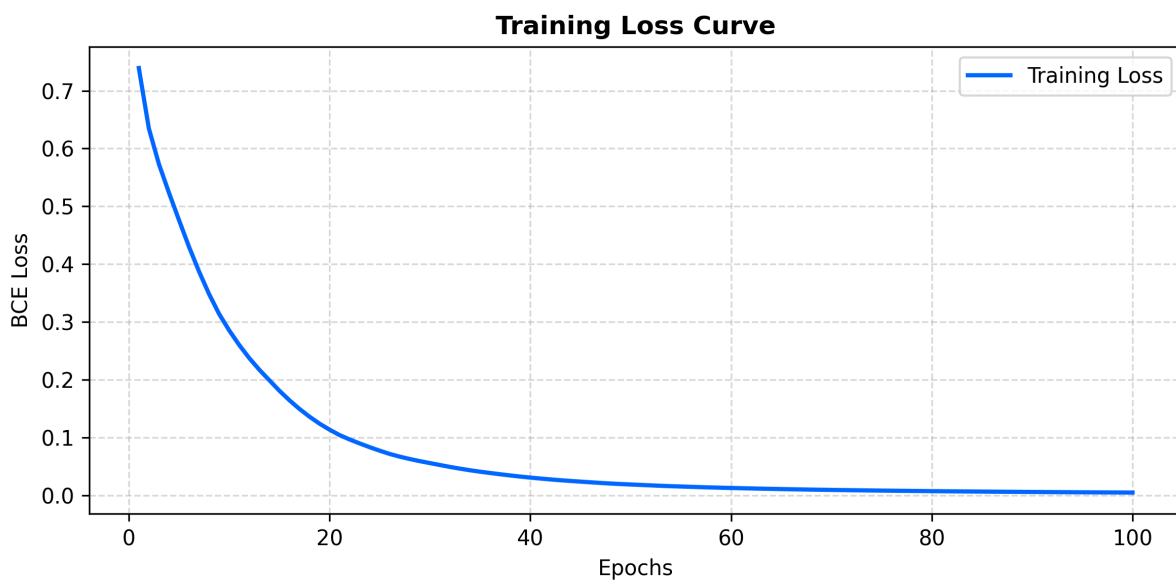


Image 2: Shows the Training Loss From the first Training Loop to the Final Training Loop where the input data is set to 10

These graphs highlights the train loss and performance of the model. As stated, during the initial training loop, its weights are random and produce inaccurate predictions, but in the final training loop, the train losses cuts down at a massive margin

We also did an evaluation where I show the metrics such as how precise the outcome is, the accuracy of the model, and the confusion matrix.

```
--- TEST SET EVALUATION METRICS ---  
Accuracy: 85.71%  
Precision: 1.0000  
Confusion Matrix:  
[[6 0]  
 [2 6]]
```

Image 1: Outcome when using the input data set of all 69 students

```
--- TEST SET EVALUATION METRICS ---  
Accuracy: 50.00%  
Precision: 0.0000  
Confusion Matrix:  
[[1 1]  
 [0 0]]
```

Image 2: Outcome when using the input data of 10 students

From Image 1, when we use all 69 students as the data set, the model performed high, which means that moved further away from making random guesses the more it's being training in the training loop. But in image 2, the model's precision and accuracy dropped significantly since it's being trained on a small number of students. However, both evaluations for accuracy are good since it's 50% or more, which means it's performing better than it randomly guessing the outcome, but the precision in image 1 is 100% because it's being trained on a larger data set, which means it's likely to not have false positives in the prediction, while image 2 is 0% because since

it's being trained on a smaller data set, it's more likely to produce false positives and render the predictions useless.

To conclude, this report shows how a simple neural network it built to predict whether a student will be promoted or retained. Using an architecture rule, the model was able to automatically scale its hidden layers based on the number of input data points. The final results show why data input size matters when building models to predict outcomes and avoid false positives. All demonstrations and program are shown on a Github Repository shown in the references.

References

Aird, U. (2026). Simple Neural Network Classification Model. Github. <https://sgu.daulric.dev/tree/main/Intro%20to%20AI/Assignment%202>